XENSIV™ TLx49012 Magnetic Position Sensor Evaluation Kit - Serial Commands Interface

Document Information

Item	Details
Document Title	Serial Commands Interface
Product	XENSIV™ TLx49012 Magnetic Position Sensor Evaluation Kit
Revision	1.0
Date	May 7, 2026
Status	Released

Revision History

Revision	Date	Author	Description
1.0	May 7, 2026	-	Initial release

Related Documents

- XENSIV™ TLx49012 Evaluation Board User Guide - Hardware setup, connection diagrams, software installation and usage etc.
- XENSIV™ TLx49012 Magnetic Position Sensor Evaluation Software - Evaluation Software
- FTDI Drivers - D2XX drivers for USB serial communication (included with software)

Scope

Purpose

This document provides a comprehensive technical reference for the serial communication protocol used by the XENSIV™ TLx49012 Magnetic Position Sensor Evaluation Kit firmware. It describes the command structure, data formats, and operational procedures required to communicate with the evaluation kit programmer via USB serial interface.

Target Audience

This documentation is intended for:

- Software developers integrating the evaluation kit into custom applications
- Test engineers developing automated test scripts
- Technical support personnel troubleshooting communication issues
- System integrators working with the TLx49012 sensor evaluation platform

Document Coverage

This document covers:

- **Bootloader Mode**: Communication protocol for firmware updates
- **Firmware Mode**: Complete command set for sensor operation, configuration, and data acquisition
- **Protocol Specifications**: Packet formats, checksums, and data encoding
- **Command Reference**: Detailed syntax and examples for all supported commands
- **Asynchronous Data**: Continuous readout packet formats for SPI and interface readouts
- **Interface Modes**: SPI, ABZ encoder, UVW Hall, PWM, and combined modes

Out of Scope

This document does NOT cover:

- Hardware design, schematics, or electrical specifications (see Evaluation Board User Guide)
- Sensor calibration procedures or magnetic field theory
- Graphical User Interface (GUI) software operation
- Application-specific implementation examples beyond basic protocol usage
- Mechanical installation or mounting guidelines
- Safety and regulatory compliance information
- Factory-level programming and calibration procedures

Prerequisites

Before using this documentation:

- Review the **XENSIV™ TLx49012 Evaluation Board User Guide** for hardware setup
- Know the mapping of the internal registers and memory of the TLx49012 sensor (available in the datasheet)
- Ensure proper USB driver installation for COM port enumeration
- Understand basic serial communication concepts (UART, baud rate, checksums)
- Have familiarity with hexadecimal notation and bitwise operations

---

Hardware Prerequisites

Hardware Connection

Before using the serial commands documented here, ensure proper hardware setup:

1. **Physical Connection**: Connect the evaluation kit programmer to your PC via USB
2. **Power Supply**: Verify the evaluation board is properly powered
3. **Sensor Installation**: Ensure the TLx49012 sensor is correctly connected to the evaluation board

Important: For detailed hardware setup instructions, connection diagrams, and safety information, please refer to the **XENSIV™ TLx49012 Evaluation Board User Guide**.

COM Port Detection

After connecting the hardware:

1. The programmer will enumerate as a virtual COM port on your system
2. Identify the COM port number from your operating system's device manager
3. Use this COM port for all serial communication as described in this document

---

Bootloader Mode

Overview

Before entering firmware mode, the MCU operates in bootloader mode. The bootloader provides basic communication validation and firmware update capabilities.

Connection Setup

Serial Port Configuration:

- **Baud Rate**: 3,000,000
- **Data Bits**: 8
- **Stop Bits**: 1
- **Parity**: None

Bootloader Sequence

Step 1: Power On

Power on the programmer device.

Step 2: Connect to COM Port

Establish serial connection using the configuration above.

Step 3: Bootloader Handshake

Sequence 1 - Echo Command:

- Send `0x09` (9 decimal) - Echo command
- MCU Response: `0x20` (32 decimal) - Bootloader acknowledgment

Sequence 2 - Switch to Firmware Mode:

- Send `0x12` (18 decimal) - Exit bootloader and start firmware
- MCU Response: `0x20` (32 decimal) - Acknowledgment before restart
- Result: MCU performs Alternative Boot Mode 0 (ABM0) restart and enters firmware mode

Bootloader Commands

Command	Decimal	Description	Response
0x09	9	Echo - Validate communication	0x20 (32)
0x10	16	Prepare page write (512 bytes)	0x20 (32)
0x11	17	Write page to flash	0x20 (32)
0x12	18	Finalize and restart to firmware	0x20 (32) then restart

Firmware Update Process

Firmware updates must be performed separately using the **Evaluation Kit Software**. The bootloader supports page-based flash programming:

1. Send echo command (0x09) for validation
2. Send prepare page command (0x10)
3. Transmit 512-byte page content
4. Send write page command (0x11) to commit to flash
5. Repeat steps 2-4 for all pages
6. Send restart command (0x12) to complete update

Note: The bootloader uses Alternative Boot Mode 0 (ABM0) with flash start address `0x08010000` (Physical Sector 1).

Transitioning to Firmware Mode

Once the bootloader sequence is complete and the MCU restarts, all subsequent commands follow the **Firmware Mode Protocol** documented below.

Firmware Mode Protocol

After successfully completing the bootloader sequence, the MCU operates in firmware mode where the following command protocol applies.

Protocol Overview

Packet Format - Command/Reply Packets (Synchronous)

- **Header**: 1 byte - Command identifier
- **Payload Size**: 2 bytes - Length of payload (little-endian)
- **Payload**: N bytes - Command data
- **Checksum**: 1 byte - Negated sum of all previous bytes

Packet Format - Data Readout Packets (Asynchronous)

- **Header**: 1 byte - Data type identifier
- **Payload**: N bytes - Data content (size inferred from header)
- **Checksum**: 1 byte - Negated sum of all previous bytes

Checksum Calculation

The checksum is calculated by summing all bytes (header + payload size + payload), then negating the result and masking to 8 bits: `checksum = (~(header + payload_size_low + payload_size_high + sum_of_payload_bytes)) & 0xFF`

Command Reference

Command Categories

Commands are organized into the following functional groups:

1. System Commands - Firmware version, echo, reboot
2. GPIO Control Commands - Power supply and programming voltage control
3. SPI Control Commands - Enable/disable SPI interface
4. Test Mode Commands - Enter/exit test mode
5. SPI In-Frame Operations - Read/write registers in-frame
6. SPI Next-Frame Operations - Read/write registers using next-frame protocol
7. Memory Operations - Read/write CMTP (OTP) memory and access rights
8. Bitmap Operations - Read user configuration bitmaps
9. Soft Reset Commands - VM and NVM soft resets
10. Readout Commands - Continuous SPI and interface readouts
11. Synchronization Commands - Trigger sync and design step configuration

---

System Commands

0x01 - CMD_GET_FW_VERSION

Description: Retrieves the firmware version information.

Request:

- Header: `0x01`
- Payload Size: `0x0000`
- Payload: None
- Example: `01 00 00 FE`

Reply:

- Payload Size: 24 bytes
- Payload Structure:
  - Byte 0: Major version
  - Byte 1: Minor version
  - Bytes 2-3: Patch version (16-bit, little-endian)
  - Bytes 4-23: Version suffix string (20 bytes, null-padded)

Example: Request: `01 00 00 FE` | Response: `01 18 00 01 02 00 00 52 43 31 00 ... [checksum]` (Version 1.2.0-RC1)

0x09 - CMD_ECHO

Description: Echo command used to validate FTDI devices. Also stops any active readout.

Request:

- Header: `0x09`
- Payload Size: `0x0000`
- Payload: None
- Example: `09 00 00 F6`

Reply:

- Payload Size: 0 bytes
- Payload: None (ACK only)

Example: Request: `09 00 00 F6` | Response: `09 00 00 F6`

Note: This command differs from the bootloader echo (0x09). In firmware mode, it follows the full packet protocol with checksum.

0x0A - CMD_BAUDRATE_SET

Description: Sets the UART baud rate to a new value. After sending acknowledgment, the firmware will switch to the new baud rate and expect a handshake command within a timeout period.

Request:

- Header: `0x0A`
- Payload Size: `0x0004`
- Payload: 4 bytes - New baud rate (32-bit, little-endian)
- Example: `0A 04 00 C0 D4 01 00 1C` (Set baud rate to 120,000)

Reply:

- Payload Size: 0 bytes
- Payload: None (ACK before baud rate change)

Example: Request: `0A 04 00 C0 D4 01 00 1C` (120,000 baud) | Response: `0A 00 00 F5` (Firmware switches baud rate after this)

Note: After receiving the reply, immediately reconfigure your serial port to the new baud rate and send CMD_BAUDRATE_HANDSHAKE (0x0B) within the timeout period to confirm the change. If the handshake is not received, the firmware may revert to the default baud rate or become unresponsive.

0x0B - CMD_BAUDRATE_HANDSHAKE

Description: Confirms the baud rate change after receiving CMD_BAUDRATE_SET (0x0A). Must be sent after CMD_BAUDRATE_SET using the new baud rate.

Request:

- Header: `0x0B`
- Payload Size: `0x0000`
- Payload: None
- Example: `0B 00 00 F4` (Handshake for baud rate change)

Reply:

- Payload Size: 0 bytes
- Payload: None (ACK only)

Example: Request: `0B 00 00 F4` (Handshake for baud rate change) | Response: `0B 00 00 F4`

Note: Always send this command immediately after switching baud rates following CMD_BAUDRATE_SET. Failure to send the handshake within the timeout period may cause communication loss.

0x0C - CMD_REBOOT_BOOTLOADER

Description: Reboots the device into bootloader mode after a 5ms delay.

Request:

- Header: `0x0C`
- Payload Size: `0x0000`
- Payload: None
- Example: `0C 00 00 F3`

Reply:

- Payload Size: 0 bytes
- Payload: None (ACK before reboot)

Example: Request: `0C 00 00 F3` | Response: `0C 00 00 F3` (Device reboots to bootloader after 5ms)

Note: Use this command to return to bootloader mode from firmware mode (e.g., for firmware updates).

GPIO Control Commands

0x0F - CMD_IO_ENABLE_SENSOR_SUPPLY

Description: Enables the sensor power supply.

Request:

- Header: `0x0F`
- Payload Size: `0x0000`
- Payload: None
- Example: `0F 00 00 F0`

Reply:

- Payload Size: 0 bytes
- Payload: None (ACK only)

Example: Request: `0F 00 00 F0` | Response: `0F 00 00 F0`

0x10 - CMD_IO_DISABLE_SENSOR_SUPPLY

Description: Disables the sensor power supply.

Request:

- Header: `0x10`
- Payload Size: `0x0000`
- Payload: None
- Example: `10 00 00 EF`

Reply:

- Payload Size: 0 bytes
- Payload: None (ACK only)

Example: Request: `10 00 00 EF` | Response: `10 00 00 EF`

0x11 - CMD_IO_ENABLE_SENSOR_V_PROG

Description: Enables the sensor programming voltage (V_PROG).

Request:

- Header: `0x11`
- Payload Size: `0x0000`
- Payload: None
- Example: `11 00 00 EE`

Reply:

- Payload Size: 0 bytes
- Payload: None (ACK only)

Example: Request: `11 00 00 EE` | Response: `11 00 00 EE`

Warning: Only enable V_PROG when programming CMTP memory. Prolonged application may damage the sensor.

0x12 - CMD_IO_DISABLE_SENSOR_V_PROG

Description: Disables the sensor programming voltage (V_PROG).

Request:

- Header: `0x12`
- Payload Size: `0x0000`
- Payload: None
- Example: `12 00 00 ED`

Reply:

- Payload Size: 0 bytes
- Payload: None (ACK only)

Example: Request: `12 00 00 ED` | Response: `12 00 00 ED`

0x13 - CMD_WAIT_V_PROG_SETTLE

Description: Waits for programming voltage to settle (blocking operation).

Request:

- Header: `0x13`
- Payload Size: `0x0000`
- Payload: None
- Example: `13 00 00 EC`

Reply:

- Payload Size: 0 bytes
- Payload: None (ACK after settling time)

Example: Request: `13 00 00 EC` | Response: `13 00 00 EC` (after settling delay)

Note: This command introduces a delay to ensure V_PROG is stable before programming operations.

SPI Control Commands

0x0D - CMD_SPI_ENABLE

Description: Enables the SPI interface for sensor communication.

Request:

- Header: `0x0D`
- Payload Size: `0x0000`
- Payload: None
- Example: `0D 00 00 F2`

Reply:

- Payload Size: 0 bytes
- Payload: None (ACK only)

Example: Request: `0D 00 00 F2` | Response: `0D 00 00 F2`

0x0E - CMD_SPI_DISABLE

Description: Disables the SPI interface.

Request:

- Header: `0x0E`
- Payload Size: `0x0000`
- Payload: None
- Example: `0E 00 00 F1`

Reply:

- Payload Size: 0 bytes
- Payload: None (ACK only)

Example: Request: `0E 00 00 F1` | Response: `0E 00 00 F1`

Note: Disable SPI when using non-SPI interface modes (ABZ, UVW, PWM).

Test Mode Commands

0x14 - CMD_SPI_ENTER_TEST_MODE

Description: Cycles sensor power and enters test mode.

Request:

- Header: `0x14`
- Payload Size: `0x0001`
- Payload:
  - Byte 0: Set value of "1".
- Example: `14 01 00 01 E9`

Reply:

- Payload Size: 4 bytes
- Payload: SPI response frame (32-bit)

Example: Request: `14 01 00 01 E9` | Response: `14 04 00 [4 bytes SPI frame] [checksum]`

Note: This command must be executed before any memory or register operations that require test mode access. This command automatically disables CRC check for bitmap after entering test mode by writing to STAT_EN_1_REG. This allows configuration changes without triggering CRC errors. Wait at least 700µs after entering test mode before issuing additional commands to ensure chip initialization is complete.

0x15 - CMD_SPI_EXIT_TEST_MODE

Description: Exits test mode by power cycling the sensor.

Request:

- Header: `0x15`
- Payload Size: `0x0000`
- Payload: None
- Example: `15 00 00 EA`

Reply:

- Payload Size: 0 bytes
- Payload: None (ACK only)

Example: Request: `15 00 00 EA` | Response: `15 00 00 EA`

Note: Execute this command before starting interface readout to ensure the IC is in normal operating mode.

0x2A - CMD_VM_SOFT_RESET_TESTMODE

Description: Performs VM (Volatile Memory) soft reset and automatically re-enters test mode with specified access rights.

⚠️ **PREREQUISITE**: The sensor/IC must be in test mode before executing this command. Use `CMD_SPI_ENTER_TEST_MODE (0x14)` first.

Request:

- Header: `0x2A`
- Payload Size: `0x0001`
- Payload:
  - Byte 0: Set value of "1".
- Example: `2A 01 00 01 D3`

Reply:

- Payload Size: 4 bytes
- Payload: SPI response frame (32-bit)

Example: Request: `2A 01 00 01 D3` | Response: `2A 04 00 [4 bytes SPI frame] [checksum]`

Note: This command performs a soft reset and automatically re-enters test mode. After execution, the sensor will be in test mode. CRC check for bitmap is automatically disabled after reset. In addition to performing the soft reset and re-entering test mode, this command also explicitly disables CRC check for bitmap by writing to STAT_EN_1_REG. Wait at least 700µs after this command before issuing additional commands.

SPI In-Frame Operations

0x16 - CMD_SPI_WRITE_IN_FRAME

Description: Writes a register value using in-frame SPI protocol.

⚠️ **PREREQUISITE**: The sensor/IC must be in test mode before executing this command. Use `CMD_SPI_ENTER_TEST_MODE (0x14)` first.

Request:

- Header: `0x16`
- Payload Size: `0x0003`
- Payload:
  - Byte 0: Register address (7-bit, 0-127)
  - Bytes 1-2: Value (16-bit, little-endian)
- Example: `16 03 00 78 CD AB F6` (Write 0xABCD to register 120)

Reply:

- Payload Size: 4 bytes
- Payload: SPI response frame (32-bit)

Example: Request: `16 03 00 78 CD AB F6` (Write 0xABCD to register 120) | Response: `16 04 00 [4 bytes SPI frame] [checksum]`

0x17 - CMD_SPI_READ_IN_FRAME

Description: Reads a register value using in-frame SPI protocol.

⚠️ **PREREQUISITE**: The sensor/IC must be in test mode before executing this command. Use `CMD_SPI_ENTER_TEST_MODE (0x14)` first.

Request:

- Header: `0x17`
- Payload Size: `0x0002`
- Payload:
  - Byte 0: Register address (7-bit, 0-127)
  - Byte 1: Clear status flag (0 = keep, 1 = clear)
- Example: `17 02 00 10 01 D5` (Read register 16, clear status)

Reply:

- Payload Size: 4 bytes
- Payload: SPI response frame (32-bit)

Example: Request: `17 02 00 10 01 D5` (Read register 16, clear status) | Response: `17 04 00 [4 bytes SPI frame] [checksum]`

SPI Next-Frame Operations

0x18 - CMD_SPI_COMMAND_NEXT_FRAME

Description: Initiates a next-frame command sequence with address and access type.

⚠️ **PREREQUISITE**: The sensor/IC must be in test mode before executing this command. Use `CMD_SPI_ENTER_TEST_MODE (0x14)` first.

Request:

- Header: `0x18`
- Payload Size: `0x0003`
- Payload:
  - Bytes 0-1: Register address (16-bit, little-endian)
  - Byte 2: Access type (0=Read Inc, 1=Write Cont, 2=Write Inc)
- Example: `18 03 00 40 01 00 A3` (Address 320, Read Inc)

Reply:

- Payload Size: 4 bytes
- Payload: SPI response frame (32-bit)

Example: Request: `18 03 00 40 01 00 A3` (Address 320, Read Inc) | Response: `18 04 00 [4 bytes SPI frame] [checksum]`

Access Type Values:

- `0b00` (0) - Read with auto-increment
- `0b01` (1) - Write continuous (same address)
- `0b10` (2) - Write with auto-increment

0x19 - CMD_SPI_END_COMMAND_NEXT_FRAME

Description: Ends a next-frame command sequence.

⚠️ **PREREQUISITE**: The sensor/IC must be in test mode before executing this command. Use `CMD_SPI_ENTER_TEST_MODE (0x14)` first.

Request:

- Header: `0x19`
- Payload Size: `0x0000`
- Payload: None
- Example: `19 00 00 E6`

Reply:

- Payload Size: 4 bytes
- Payload: SPI response frame (32-bit)

Example: Request: `19 00 00 E6` | Response: `19 04 00 [4 bytes SPI frame] [checksum]`

0x1A - CMD_SPI_READ_NEXT_FRAME

Description: Reads data in a next-frame sequence (after command initiation).

⚠️ **PREREQUISITE**: The sensor/IC must be in test mode before executing this command. Use `CMD_SPI_ENTER_TEST_MODE (0x14)` first.

Request:

- Header: `0x1A`
- Payload Size: `0x0001`
- Payload:
  - Byte 0: Clear status flag (0 = keep, 1 = clear)
- Example: `1A 01 00 01 E3` (Clear status)

Reply:

- Payload Size: 4 bytes
- Payload: SPI response frame (32-bit)

Example: Request: `1A 01 00 01 E3` | Response: `1A 04 00 [4 bytes SPI frame] [checksum]`

0x1B - CMD_SPI_WRITE_NEXT_FRAME

Description: Writes data in a next-frame sequence (after command initiation).

⚠️ **PREREQUISITE**: The sensor/IC must be in test mode before executing this command. Use `CMD_SPI_ENTER_TEST_MODE (0x14)` first.

Request:

- Header: `0x1B`
- Payload Size: `0x0002`
- Payload:
  - Bytes 0-1: Value (16-bit, little-endian)
- Example: `1B 02 00 34 12 9C` (Write 0x1234)

Reply:

- Payload Size: 4 bytes
- Payload: SPI response frame (32-bit)

Example: Request: `1B 02 00 34 12 9C` (Write 0x1234) | Response: `1B 04 00 [4 bytes SPI frame] [checksum]`

0x1D - CMD_SPI_READ_REG_NEXT_FRAME

Description: Reads a specific register using next-frame protocol (single command).

⚠️ **PREREQUISITE**: The sensor/IC must be in test mode before executing this command. Use `CMD_SPI_ENTER_TEST_MODE (0x14)` first.

Request:

- Header: `0x1D`
- Payload Size: `0x0003`
- Payload:
  - Bytes 0-1: Register address (16-bit, little-endian)
  - Byte 2: Clear status flag (0 = keep, 1 = clear)
- Example: `1D 03 00 10 00 01 CE` (Read register 16, clear status)

Reply:

- Payload Size: 4 bytes
- Payload: SPI response frame (32-bit)

Example: Request: `1D 03 00 10 00 01 CE` (Read register 16, clear status) | Response: `1D 04 00 [4 bytes SPI frame] [checksum]`

0x1E - CMD_SPI_WRITE_REG_NEXT_FRAME

Description: Writes a specific register using next-frame protocol (single command).

⚠️ **PREREQUISITE**: The sensor/IC must be in test mode before executing this command. Use `CMD_SPI_ENTER_TEST_MODE (0x14)` first.

Request:

- Header: `0x1E`
- Payload Size: `0x0004`
- Payload:
  - Bytes 0-1: Register address (16-bit, little-endian)
  - Bytes 2-3: Value (16-bit, little-endian)
- Example: `1E 04 00 10 00 34 12 87` (Write 0x1234 to register 16)

Reply:

- Payload Size: 4 bytes
- Payload: SPI response frame (32-bit)

Example: Request: `1E 04 00 10 00 34 12 87` (Write 0x1234 to register 16) | Response: `1E 04 00 [4 bytes SPI frame] [checksum]`

Memory Operations

0x1C - CMD_SPI_READ_CMTP

Description: Reads all CMTP (OTP) memory contents.

⚠️ **PREREQUISITE**: The sensor/IC must be in test mode before executing this command. Use `CMD_SPI_ENTER_TEST_MODE (0x14)` first.

Request:

- **Header**: `0x1C`
- **Payload Size**: `0x0000`
- **Payload**: None

Example: `1C 00 00 E3`

Reply:

- **Payload Size**: 120 bytes (60 words)
- **Payload**: Memory contents as 16-bit values (little-endian)

Example:

- Request: `1C 00 00 E3` (Read CMTP memory)
- Response: `1C 78 00 [60 words of memory] [checksum]` (120 bytes = 60x 16-bit words)

Notes:

- Reads memory from addresses 256-315 (0x0100-0x013B)
- Returns 60 words (120 bytes) of CMTP data
- Data format: Little-endian 16-bit words

0x28 - CMD_READ_ALL_ACCESS_RIGHTS

Description: Reads the remaining programming cycles for all CMTP memory locations. For each CMTP line, the command reads OTP status register 254. The number of remaining programming cycles is extracted from bits 12:8 of register 254.

⚠️ **PREREQUISITE**: The sensor/IC must be in test mode before executing this command. Use `CMD_SPI_ENTER_TEST_MODE (0x14)` first.

Request:

- **Header**: `0x28`
- **Payload Size**: `0x0000`
- **Payload**: None

Example: `28 00 00 D7`

Reply:

- **Payload Size**: 120 bytes (60 words)
- **Payload**: Access rights data as 16-bit values (little-endian)
  - Each 16-bit value contains OTP_STAT_REG[254] content for the corresponding address
  - Bits 12:8 of each word represent the number of remaining programming cycles

Example:

- Request: `28 00 00 D7` (Read access rights)
- Response: `28 78 00 [60 words] [checksum]`

Access Rights Data Processing:

For each 16-bit word in the response, extract the remaining programming cycles using:

```
cycles = (word >> 8) & 0x1F;
```

0x29 - CMD_WRITE_CMTP_LINE

Description: Programs one line (address-value pair) to CMTP memory. Applies programming voltage. Address must be within valid range.

⚠️ **PREREQUISITE**: The sensor/IC must be in test mode before executing this command. Use `CMD_SPI_ENTER_TEST_MODE (0x14)` first.

⚠️ **WARNING**: Ensure the pre-programming workflow has been done before using this command. Ensure the programming voltage (V_PROG) is enabled and settled before executing this command.

Request:

- **Header**: `0x29`
- **Payload Size**: `0x0004`
- **Payload**:
  - Bytes 0-1: Address (16-bit, little-endian)
  - Bytes 2-3: Value (16-bit, little-endian)

Example: `29 04 00 00 01 CD AB 59` (Write 0xABCD to address 256)

Reply:

- **Payload Size**: 3 bytes
- **Payload**:
  - Byte 0: Dummy status
  - Byte 1: OTP status register 254 error (0 = OK)
  - Byte 2: OTP status register 255 error (0 = OK)

Example:

- Request: `29 04 00 00 01 CD AB 59` (Write 0xABCD to address 256)
- Response: `29 03 00 01 00 00 D2` (Status: 0x00, 0x00 - NO error)

Status Interpretation:

- `0x00, 0x00` - Successful write
- Non-zero values indicate programming errors

Bitmap Operations

0x25 - CMD_SPI_READ_BM_USR_CONFIG

Description: Reads the user configuration bitmap from sensor volatile memory.

⚠️ **PREREQUISITE**: The sensor/IC must be in test mode before executing this command. Use `CMD_SPI_ENTER_TEST_MODE (0x14)` first.

Request:

- **Header**: `0x25`
- **Payload Size**: `0x0000`
- **Payload**: None

Example: `25 00 00 DA`

Reply:

- **Payload Size**: Variable (depends on design step)
- **Payload**: User configuration bitmap data (16-bit words, little-endian)

Example:

- Request: `25 00 00 DA`
- Response: `25 [size] 00 [bitmap data] [checksum]`

Notes:

- Bitmap size varies by design step
- Contains user-configurable parameters
- Includes CRC at the end of the bitmap

Soft Reset Commands

0x21 - CMD_VM_SOFT_RESET

Description: Performs a soft reset of the Volatile Memory (VM) in the sensor.

⚠️ **PREREQUISITE**: The sensor/IC must be in test mode before executing this command. Use `CMD_SPI_ENTER_TEST_MODE (0x14)` first.

Request:

- **Header**: `0x21`
- **Payload Size**: `0x0000`
- **Payload**: None

Example: `21 00 00 DE`

Reply:

- **Payload Size**: 0 bytes
- **Payload**: None (ACK only)

Example:

- Request: `21 00 00 DE`
- Response: `21 00 00 DE`

Note: VM soft reset reloads configuration from CMTP into volatile registers.

0x22 - CMD_NVM_SOFT_RESET

Description: Performs a soft reset of the Non-Volatile Memory (NVM) in the sensor.

⚠️ **PREREQUISITE**: The sensor/IC must be in test mode before executing this command. Use `CMD_SPI_ENTER_TEST_MODE (0x14)` first.

Request:

- **Header**: `0x22`
- **Payload Size**: `0x0000`
- **Payload**: None

Example: `22 00 00 DD`

Reply:

- **Payload Size**: 0 bytes
- **Payload**: None (ACK only)

Example:

- Request: `22 00 00 DD`
- Response: `22 00 00 DD`

Readout Commands

SPI Register Readout

0x1F - CMD_START_ASYNC_REG_READOUT

Description: Starts continuous readout of a specific SPI register.

⚠️ **PREREQUISITE**: The sensor/IC must be in test mode before executing this command. Use `CMD_SPI_ENTER_TEST_MODE (0x14)` first.

Request:

- **Header**: `0x1F`
- **Payload Size**: `0x0004`
- **Payload**:
  - Bytes 0-1: Register address (16-bit, little-endian)
  - Byte 2: Clear status flag (0 = keep, 1 = clear)
  - Byte 3: In-frame mode (0 = next-frame, 1 = in-frame)

Example: `1F 04 00 10 00 01 01 CA` (Read register 16, clear status, in-frame mode)

Reply:

- **Payload Size**: 0 bytes
- **Payload**: None (ACK only)
- **Asynchronous data packets** (header `0x64`) will be sent continuously

Example:

- Request: `1F 04 00 10 00 01 01 CA`
- Response: `1F 00 00 E0` (Async data packets with header 0x64 follow...)

Note: See **0x64 - CMD_SPI_ASYNC_REG** for packet format.

0x20 - CMD_STOP_ASYNC_REG_READOUT

Description: Stops continuous SPI register readout.

Request:

- **Header**: `0x20`
- **Payload Size**: `0x0000`
- **Payload**: None

Example: `20 00 00 DF`

Reply:

- **Payload Size**: 0 bytes
- **Payload**: None (ACK only)

Example:

- Request: `20 00 00 DF`
- Response: `20 00 00 DF`

SPI Register Group Readout

0x23 - CMD_START_ASYNC_REG_GROUP_READOUT

Description: Starts continuous readout of a predefined group of SPI registers.

⚠️ **PREREQUISITE**: The sensor/IC must be in test mode before executing this command. Use `CMD_SPI_ENTER_TEST_MODE (0x14)` first.

Request:

- **Header**: `0x23`
- **Payload Size**: `0x0003`
- **Payload**:
  - Byte 0: Data selection group (see table below)
  - Byte 1: Clear status flag (0 = keep, 1 = clear)
  - Byte 2: In-frame mode (0 = next-frame, 1 = in-frame)

Example: `23 03 00 01 01 01 D6` (Angle/Speed group, clear status, in-frame)

Reply:

- **Payload Size**: 0 bytes
- **Payload**: None (ACK only)
- **Asynchronous data packets** will be sent continuously (header depends on data selection)

Data Selection Groups:

Data Selection Value	Group	Async Header	Description
1	Angle/Speed	0x65 (101)	Angle and velocity registers
2	Vector Length	0x66 (102)	Vector length data
3	X/Y Components	0x67 (103)	X and Y magnetic field components
4	System Calibration	0x68 (104)	System calibration registers

Example:

- Request: `23 03 00 01 01 01 D6` (Angle/Speed group)
- Response: `23 00 00 DC` (Async data packets with header 0x65 follow...)

0x24 - CMD_STOP_ASYNC_REG_GROUP_READOUT

Description: Stops continuous SPI register group readout.

Request:

- **Header**: `0x24`
- **Payload Size**: `0x0000`
- **Payload**: None

Example: `24 00 00 DB`

Reply:

- **Payload Size**: 0 bytes
- **Payload**: None (ACK only)

Example:

- Request: `24 00 00 DB`
- Response: `24 00 00 DB`

Interface Readout

0x2C - CMD_START_IF_READOUT

Description: Starts continuous readout of the configured interface output (SPI, ABZ, UVW, PWM, or combinations).

⚠️ **IMPORTANT**: This command can be called from either test mode or normal mode. When called from test mode (which is required for VM configuration workflow), the command internally performs the necessary transitions to normal operating mode before starting interface readout. Executing this command will take the sensor out of Test Mode automatically.

Request:

- **Header**: `0x2C`
- **Payload Size**: `0x0009`
- **Payload**:
  - Byte 0: Interface mode (0=SPI, 1=ABZ, 2=UVW, 3=PWM, 4=ABZ+PWM, 5=UVW+PWM)
  - Byte 1: PWM frequency index (0=250Hz, 1=500Hz, 2=1000Hz, 3=2000Hz, 4=2500Hz, 5=3000Hz, 6=3500Hz, 7=4000Hz)
  - Byte 2: PWM start edge (0=falling, 1=rising)
  - Byte 3: ABZ mode (0=AB, 1=Step/Direction)
  - Byte 4: ABZ initial position mode (0=enabled, 1=disabled)
  - Bytes 5-6: ABZ pulses per revolution (16-bit, little-endian)
  - Byte 7: UVW pole pairs (1-15)
  - Byte 8: ABZ Z-pulse mode (0=ungated, 1=AB-gated, 2=A-gated, 3=B-gated)

Example: `2C 09 00 01 03 00 00 00 FF 0F 0F 00 A9` (ABZ mode, 4096 pulses, A/B mode, Ungated tz)

Reply:

- **Payload Size**: 0 bytes
- **Payload**: None (ACK only)
- **Asynchronous data packets** will be sent continuously (header depends on interface mode)

Interface Modes:

Value	Mode	Async Header	Description
0	SPI	0x64-0x68	SPI register readout
1	ABZ	0x69 (105)	ABZ encoder output
2	UVW	0x6B (107)	UVW Hall sensor output
3	PWM	0x6A (106)	PWM output
4	ABZ_PWM	0x69, 0x6A	Combined ABZ and PWM
5	UVW_PWM	0x6B, 0x6A	Combined UVW and PWM

Example:

- Request: `2C 09 00 01 03 00 00 00 FF 0F 0F 00 A9` (ABZ, 4096, A/B mode)
- Response: `2C 00 00 D3` (Async data packets with header 0x69 follow...)

0x2D - CMD_STOP_IF_READOUT

Description: Stops continuous readout of the configured interface.

Request:

- **Header**: `0x2D`
- **Payload Size**: `0x0000`
- **Payload**: None

Example: `2D 00 00 D2`

Reply:

- **Payload Size**: 0 bytes
- **Payload**: None (ACK only)

Example:

- Request: `2D 00 00 D2`
- Response: `2D 00 00 D2`

Synchronization Commands

0x27 - CMD_TRIGGER_SYNC_IFE

Description: Triggers a synchronization pulse on the IFE (Interface Enable) pin.

Request:

- **Header**: `0x27`
- **Payload Size**: `0x0001`
- **Payload**:
  - Byte 0: Edge type (1 = rising edge, 2 = falling edge, 3 = any edge)

Example: `27 01 00 01 D6` (Rising edge)

Reply:

- **Payload Size**: 0 bytes
- **Payload**: None (ACK only)

Example:

- Request: `27 01 00 01 D6` (Rising edge)
- Response: `27 00 00 D8`

Note: Used to synchronize sensor angle updates with external events. Can be used in both test mode and normal operating mode.

0x2B - CMD_SET_DESIGN_STEP

Description: Configures the MCU for a specific sensor design step variant.

Request:

- **Header**: `0x2B`
- **Payload Size**: `0x0001`
- **Payload**:
  - Byte 0: Design step (0=A11, 1=B11, 2=B12)

Example: `2B 01 00 01 D2` (Design step B11)

Reply:

- **Payload Size**: 0 bytes
- **Payload**: None (ACK only)

Example:

- Request: `2B 01 00 01 D2` (Design step B11)
- Response: `2B 00 00 D4`

Design Step Values:

Value	Design Step	Description
0	A11	Initial design step
1	B11	Second design iteration
2 (default)	B12	Third design iteration

Note: This command configures the MCU's internal design step setting, which determines memory map layouts and register definitions. This command should be executed early in the initialization sequence. Does not require sensor to be in test mode.

Asynchronous Packets

Asynchronous packets are sent by the MCU without a preceding request when continuous readout modes are active. These packets do not include the payload size field - the size is inferred from the header type.

0x64 (100) - CMD_SPI_ASYNC_REG

Description: Asynchronous packet containing a single SPI register readout. Sent continuously when single register readout is active (started by `CMD_START_ASYNC_REG_READOUT` 0x1F).

Packet Structure:

- **Header**: `0x64`
- **Payload**: 4 bytes (32-bit SPI frame)
- **Checksum**: 1 byte

Format: `64 [4 bytes SPI frame] [checksum]`

Payload Structure (SPI Frame):

- Bytes 0-3: 32-bit SPI response frame
  - Status bits
  - Register data (16-bit)
  - Protocol bits

Example: `64 12 34 56 78 [checksum]`

Notes:

- Triggered by PWM-based periodic timer during active register readout
- Update rate depends on configured readout period
- Requires IC to be in test mode

0x65 (101) - CMD_SPI_ASYNC_ANG_SPD

Description: Asynchronous packet containing angle and speed data. Sent periodically when angle/speed group readout is active (started by `CMD_START_ASYNC_REG_GROUP_READOUT` 0x23 with data selection 1).

Packet Structure:

- **Header**: `0x65`
- **Payload**: 8 bytes
- **Checksum**: 1 byte

Format: `65 [8 bytes] [checksum]`

Payload Structure:

- Bytes 0-3: Angle SPI frame (32-bit)
- Bytes 4-7: Velocity SPI frame (32-bit)

Example: `65 [angle frame] [velocity frame] [checksum]`

Notes:

- Contains both angle and velocity register data
- Useful for motion control applications
- Requires IC to be in test mode

0x66 (102) - CMD_SPI_ASYNC_VECT_LGTH

Description: Asynchronous packet containing vector length data. Sent periodically when vector length group readout is active (started by `CMD_START_ASYNC_REG_GROUP_READOUT` 0x23 with data selection 2).

Packet Structure:

- **Header**: `0x66`
- **Payload**: 8 bytes
- **Checksum**: 1 byte

Format: `66 [8 bytes] [checksum]`

Payload Structure:

- Bytes 0-3: Vector length frame 1 (32-bit)
- Bytes 4-7: Vector length frame 2 (32-bit)

Example: `66 [vector data 1] [vector data 2] [checksum]`

Notes:

- Vector length indicates magnetic field strength
- Useful for airgap monitoring and diagnostics
- Requires IC to be in test mode

0x67 (103) - CMD_SPI_ASYNC_X_Y

Description: Asynchronous packet containing X and Y magnetic field components and related data. Sent periodically when X/Y group readout is active (started by `CMD_START_ASYNC_REG_GROUP_READOUT` 0x23 with data selection 3).

Packet Structure:

- **Header**: `0x67`
- **Payload**: 32 bytes
- **Checksum**: 1 byte

Format: `67 [32 bytes] [checksum]`

Payload Structure:

- Bytes 0-31: Multiple SPI frames containing X, Y, and related magnetic field data

Example: `67 [32 bytes of XY data] [checksum]`

Notes:

- Contains comprehensive magnetic field vector information
- Useful for calibration and field analysis
- Requires IC to be in test mode

0x68 (104) - CMD_SPI_ASYNC_SYS_CAL

Description: Asynchronous packet containing system calibration register data. Sent periodically when system calibration group readout is active (started by `CMD_START_ASYNC_REG_GROUP_READOUT` 0x23 with data selection 4).

Packet Structure:

- **Header**: `0x68`
- **Payload**: 12 bytes
- **Checksum**: 1 byte

Format: `68 [12 bytes] [checksum]`

Payload Structure:

- Bytes 0-11: System calibration register frames

Example: `68 [12 bytes calibration data] [checksum]`

Notes:

- Contains factory and runtime calibration parameters
- Useful for diagnostics and verification
- Requires IC to be in test mode

0x69 (105) - CMD_IF_POSIF_ENCODER

Description: Asynchronous packet containing ABZ encoder interface data. Sent continuously when ABZ interface readout is active (started by `CMD_START_IF_READOUT` 0x2C with interface mode 1 or 4).

Packet Structure:

- **Header**: `0x69`
- **Payload**: 16 bytes
- **Checksum**: 1 byte

Format: `69 [16 bytes] [checksum]`

Payload Structure: - Bytes 0-15: 4 unused Bytes, Counted Pulses, unused Bytes

- 4 unused bytes (can be ignored)
- 4 bytes: Position counter value (32-bit, little-endian)
- 4 bytes of the set ticksPerRevolution (that was sent during Interface setup)
- 4 unused bytes (can be ignored)

Example: `69 00 00 00 00 7D 3C 00 00 00 40 00 00 00 00 00 00 9D` where `7D 3C 00 00` is the position counter value (Little-endian). **Notes:**

- Contains only ABZ encoder pulses count.
- Requires IC to be in normal operating mode (NOT test mode)

0x6A (106) - CMD_IF_PWM

Description: Asynchronous packet containing PWM interface data. Sent continuously when PWM interface readout is active (started by `CMD_START_IF_READOUT` 0x2C with interface mode 3, 4, or 5).

Packet Structure:

- **Header**: `0x6A`
- **Payload**: 10 bytes
- **Checksum**: 1 byte

Format: `6A [10 bytes] [checksum]`

Payload Structure:

- Bytes 0-9: PWM timing and duty cycle data
  - 4 bytes of how many ticks passed since the last PWM period (32-bit, little-endian). It is only useful if the microcontroller clock speed is known. (used for PWM frequency)
  - 4 bytes of how many ticks passed since the last PWM edge (32-bit, little-endian). It is only useful if the microcontroller clock speed is known. (used for PWM duty cycle)
  - 2 bytes of the configured PWM starting Edge

Example: `6A 88 E0 00 00 04 48 00 00 00 00 E1`

Notes:

- Contains PWM output timing measurements
- Update rate matches configured PWM frequency
- Requires IC to be in normal operating mode (NOT test mode)

0x6B (107) - CMD_IF_HALL

Description: Asynchronous packet containing UVW Hall sensor interface data. Sent continuously when UVW interface readout is active (started by `CMD_START_IF_READOUT` 0x2C with interface mode 2 or 5).

Packet Structure:

- **Header**: `0x6B`
- **Payload**: 6 bytes
- **Checksum**: 1 byte

Format: `6B [6 bytes] [checksum]`

Payload Structure:

- Bytes 0-5: UVW Hall sensor state and commutation data
  - 1 byte of current Hall state (bits 0-2: U, V, W state; bits 3-7: unused)
  - 4 bytes of signed value representing the number of electrical rotations
  - 1 byte of set pole pairs (that was sent during Interface setup)

Example: `6B 03 38 00 00 00 10 49`

Notes:

- Contains UVW Hall effect sensor output for motor commutation
- Update rate depends on pole pairs and rotation speed
- Requires IC to be in normal operating mode (NOT test mode)

Command Summary Table

Hex	Dec	Command	Category	Test Mode Required
0x01	1	CMD_GET_FW_VERSION	System	No
0x09	9	CMD_ECHO	System	No
0x0A	10	CMD_BAUDRATE_SET	System	No
0x0B	11	CMD_BAUDRATE_HANDSHAKE	System	No
0x0C	12	CMD_REBOOT_BOOTLOADER	System	No
0x0D	13	CMD_SPI_ENABLE	SPI Control	No
0x0E	14	CMD_SPI_DISABLE	SPI Control	No
0x0F	15	CMD_IO_ENABLE_SENSOR_SUPPLY	GPIO Control	No
0x10	16	CMD_IO_DISABLE_SENSOR_SUPPLY	GPIO Control	No
0x11	17	CMD_IO_ENABLE_SENSOR_V_PROG	GPIO Control	No
0x12	18	CMD_IO_DISABLE_SENSOR_V_PROG	GPIO Control	No
0x13	19	CMD_WAIT_V_PROG_SETTLE	Timing	No
0x14	20	CMD_SPI_ENTER_TEST_MODE	Test Mode	-
0x15	21	CMD_SPI_EXIT_TEST_MODE	Test Mode	-
0x16	22	CMD_SPI_WRITE_IN_FRAME	SPI In-Frame	Yes
0x17	23	CMD_SPI_READ_IN_FRAME	SPI In-Frame	Yes
0x18	24	CMD_SPI_COMMAND_NEXT_FRAME	SPI Next-Frame	Yes
0x19	25	CMD_SPI_END_COMMAND_NEXT_FRAME	SPI Next-Frame	Yes
0x1A	26	CMD_SPI_READ_NEXT_FRAME	SPI Next-Frame	Yes
0x1B	27	CMD_SPI_WRITE_NEXT_FRAME	SPI Next-Frame	Yes
0x1C	28	CMD_SPI_READ_CMTP	Memory	Yes
0x1D	29	CMD_SPI_READ_REG_NEXT_FRAME	SPI Next-Frame	Yes
0x1E	30	CMD_SPI_WRITE_REG_NEXT_FRAME	SPI Next-Frame	Yes
0x1F	31	CMD_START_ASYNC_REG_READOUT	Readout	Yes
0x20	32	CMD_STOP_ASYNC_REG_READOUT	Readout	No
0x21	33	CMD_VM_SOFT_RESET	Soft Reset	Yes
0x22	34	CMD_NVM_SOFT_RESET	Soft Reset	Yes
0x23	35	CMD_START_ASYNC_REG_GROUP_READOUT	Readout	Yes

Hex	Dec	Command	Category	Test Mode Required
0x24	36	CMD_STOP_ASYNC_REG_GROUP_READOUT	Readout	No
0x25	37	CMD_SPI_READ_BM_USR_CONFIG	Bitmap	Yes
0x27	39	CMD_TRIGGER_SYNC_IFE	Synchronization	No
0x28	40	CMD_READ_ALL_ACCESS_RIGHTS	Memory	Yes
0x29	41	CMD_WRITE_CMTP_LINE	Memory	Yes
0x2A	42	CMD_VM_SOFT_RESET_TESTMODE	Soft Reset	Yes
0x2B	43	CMD_SET_DESIGN_STEP	Configuration	No
0x2C	44	CMD_START_IF_READOUT	Readout	Works from test or normal mode
0x2D	45	CMD_STOP_IF_READOUT	Readout	No

Volatile Memory Configuration Workflow

This section provides a step-by-step workflow for configuring the sensor's interface mode (ABZ, UVW, PWM) at runtime without permanently programming CMTP memory.

Overview

The TLx49012 sensor interface reconfiguration involves **two stages**:

Stage 1: Configure Sensor's Volatile Memory (VM)

- Write interface configuration to sensor's user config registers
- Modify registers 63 and 64 to set interface mode, resolution, frequency, etc.
- Calculate and write CRC to register 77
- Soft reset to apply configuration

Stage 2: Configure MCU to Read Interface Signals

- Use `CMD_START_IF_READOUT (0x2C)` to configure the MCU/firmware
- MCU reconfigures peripherals (POSIF, PWM, HALL) to capture interface signals
- Starts continuous readout and forwarding of interface data via USB

This two-stage approach is necessary because:

- The **sensor** generates the interface signals (ABZ pulses, UVW states, PWM output)
- The **MCU** captures those signals and forwards them to the PC via USB

⚠️ **CRITICAL WARNING**: Do **NOT** exit test mode after writing VM configuration! Exiting test mode performs a power cycle that resets VM and **erases all configuration changes**. The sensor must remain in test mode after Step 10 (soft reset), and `CMD_START_IF_READOUT` will handle the transition to normal operating mode automatically.

This workflow is useful for:

- Testing different interface modes without committing changes to OTP
- Prototyping and development before finalizing configuration
- Dynamic reconfiguration based on application needs

Changes to VM are temporary and will be lost on power cycle or sensor reset unless they are also written to CMTP memory.

Prerequisites

- Evaluation kit connected to PC via USB
- FTDI drivers installed (COM port available)
- Sensor properly connected and powered
- Understanding of bitmap structure and register bitfield layout
- Knowledge of CRC8 SAE J1850 calculation (seed 0xAA)

Interface Mode Configuration Flow

This workflow demonstrates the complete two-stage process for configuring interface modes.

STAGE 1: Configure Sensor's Volatile Memory

Step 1: Connect Programmer to PC

1. Connect the evaluation kit programmer to your PC via USB
2. Wait for the device to enumerate as a COM port
3. Identify the COM port number from Device Manager (e.g., COM3, COM5)

Step 2: Configure Serial Port

Open the serial port with the following configuration:

- **Baud Rate**: 3,000,000
- **Data Bits**: 8
- **Stop Bits**: 1
- **Parity**: None

Step 3: Execute Bootloader Commands

Perform the bootloader handshake sequence to transition to firmware mode.

3.1 - Send Echo Command: Send `0x09` and wait for response `0x20` (bootloader acknowledgment).

3.2 - Switch to Firmware Mode: Send `0x12` and wait for response `0x20`. MCU will restart into firmware mode.

Note: After restart, wait at least 100ms for firmware initialization.

Step 4: Enable Sensor Supply and SPI

4.1 - Enable Sensor Supply: Send `0F 00 00 F0` (CMD_IO_ENABLE_SENSOR_SUPPLY) and wait for response `0F 00 00 F0`.

4.2 - Enable SPI Interface: Send `0D 00 00 F2` (CMD_SPI_ENABLE) and wait for response `0D 00 00 F2`.

Step 5: Enter Test Mode

Enter test mode to access sensor registers. Send `14 01 00 01 E9` (CMD_SPI_ENTER_TEST_MODE with access rights 0x01) and wait for response `14 04 00 [4 bytes SPI frame] [checksum]`. Sensor is now in test mode. CRC check for bitmap is automatically disabled.

Note: Wait at least 700µs after entering test mode before proceeding.

Step 6: Set Design Step

⚠️ **MANDATORY**: Configure the MCU for the correct design step. Send `2B 01 00 02 D1` (CMD_SET_DESIGN_STEP - Design step B12, value 2) and wait for response `2B 00 00 D4`. This configures the MCU's register map and memory layout for the sensor variant.

Note: This step is required before any register read/write operations.

Step 7: Read Current User Configuration Registers

Read **all 15 user configuration registers** (addresses 63-77 / 0x3F-0x4D).

⚠️ **IMPORTANT**: You must read all 15 registers, as all of them will be needed for CRC calculation.

7.1 - Read All Registers (63-77): For each register from 63 to 77, execute: Send `1D 03 00 [addr_low] [addr_high] 01 [checksum]` and wait for `1D 04 00 [4 bytes SPI response] [checksum]`. Extract the 16-bit register value from SPI response.

7.2 - Store All Values: Store all 15 register values for later modification and CRC calculation. These will be used as the base for writing back in Step 9.

Step 8: Modify Configuration Registers

8.1 - Modify Register 63 (0x3F) - usr_config_1_reg:

Register 63 contains the primary interface configuration bitfields.

Register 63 (0x3F) Bit Layout:

```
Bit 15: adc_max_rng_en (1 bit) - ADC max range enable
Bit 14: abz_init_pos_mode (1 bit) - ABZ initial position
Bit 13: abz_mode (1 bit) - ABZ mode
Bit 12: spi_miso_crc_inv_dis (1 bit) - SPI MISO CRC invert disable
Bit 11: spi_mosi_crc_dis (1 bit) - SPI MOSI CRC disable
Bits 10-9: sync_edge (2 bits) - Synchronization edge
Bits 8-7: sync_mode (2 bits) - Synchronization mode
Bit 6: pwm_start_edge (1 bit) - PWM starting edge
Bits 5-3: pwm_freq (3 bits) - PWM frequency
Bits 2-0: if_mode (3 bits) - Interface mode
```

Calculate the new register 63 value based on your target interface mode. Preserve other bitfields or use known working values.

8.2 - Modify Register 64 (0x40) - usr_config_2_reg:

Register 64 contains ABZ/UVW resolution and Z-pulse configuration.

Register 64 (0x40) Bit Layout:

```
Bits 15-14: cfg_unused_64
Bits 13-12: abz_z_mode - ABZ Z-pulse mode
Bits 11-0: abz_uvw_pulses_pp - ABZ pulses or UVW pole pairs

           For ABZ: value = (pulses_per_rev - 1), range 0-4095 for 1-4096 pulses
           For UVW: lower 4 bits = (pole_pairs - 1), range 0-15 for 1-16 pairs
```

Calculate the new register 64 value based on your target configuration.

8.3 - Recalculate CRC (0x4D) - usr_config_15_reg:

⚠️ **CRITICAL**: The CRC must be calculated **before** writing any registers, as it will be written as part of register 77 in Step 8.

Understanding the CRC Input Data:

The CRC calculation for user configuration uses **29 bytes** of input data:

- **Bytes 0-27**: Complete 16-bit values from registers 63-76 (14 registers × 2 bytes = 28 bytes)
- **Byte 28**: Upper byte (MSB) from register 77 (bits 15:8 only)

CRC Calculation Process:

```
Algorithm: CRC8 SAE J1850
Polynomial: 0x1D (x^8 + x^4 + x^3 + x^2 + 1)
Seed (Initial Value): 0xAA
Input Data: 29 bytes total
Output: 8-bit CRC value
```

Step 9: Write Configuration Registers

9.1 - Write the changed Registers: Write the changed registers from 63 to 76 using the CMD_SPI_WRITE_REG_NEXT_FRAME command:

```
For each register from 63 to 76:
  Send: 1E 04 00 [addr_low] [addr_high] [value_low] [value_high] [checksum]
  Wait: 1E 04 00 [4 bytes SPI response] [checksum]
```

9.2 - Update Register 77 with CRC: Register 77 contains the CRC in bits 7:0. Preserve the upper byte (bits 15:8):

```
New Register 77 = (current_reg_77 & 0xFF00) | calculated_crc
```

Write register 77 with the updated CRC value as the last register in the sequence above.

Step 10: Apply Configuration Using Soft Reset Command

Perform VM soft reset to reload configuration and re-enter test mode. Send `2A 01 00 01 D3` (CMD_VM_SOFT_RESET_TESTMODE with access rights 0x01) and wait for response `2A 04 00 [4 bytes SPI frame] [checksum]`. This command reloads volatile memory from modified registers, automatically re-enters test mode, and disables CRC check for bitmap.

Note: Wait at least 700µs after reset.

Step 11: Verify Configuration (Optional but Recommended)

Read back all modified registers to confirm the writes were successful.

11.1 - Re-read All Registers (63-77): Use the same read sequence from Step 7.

11.2 - Verify Values: Compare read values with the values you wrote in Step 9. Verify register 77 contains the correct CRC. If any mismatches are found, repeat Step 9 for those registers.

Step 12: Start Interface Readout

Use `CMD_START_IF_READOUT (0x2C)` to configure the MCU/firmware to capture and forward interface signals.

Command Behavior: The `CMD_START_IF_READOUT` command configures the MCU to: (1) Perform VM soft reset (for non-SPI modes), (2) Disable SPI interface (for non-SPI modes), (3) Reconfigure MCU peripherals (POSIF, PWM, HALL) to capture the sensor's interface signals, (4) Start continuous readout and forwarding of interface data via Serial.

Step 13: Verify Interface Output

13.1 - Observe Async Data Packets: Monitor the async packets based on interface mode: ABZ Mode (Header `0x69` - Encoder position, direction, timing), UVW Mode (Header `0x6B` - Hall sensor states, commutation sector), PWM Mode (Header `0x6A` - PWM period, duty cycle).

13.2 - Check Functionality: Verify the interface behaves as expected: ABZ (Position counter increments/decrements, direction changes, Z-pulse behavior), UVW (Correct sector transitions, pole pair configuration), PWM (Correct frequency and duty cycle encoding).

Step 14: Stop Readout (Optional)

When testing is complete or to switch modes, send `2D 00 00 D2` (CMD_STOP_IF_READOUT) and wait for response `2D 00 00 D2`.

Note: To test a different configuration, repeat from Step 5 (Enter Test Mode) in Stage 1.

Complete Examples:

1. Configuring ABZ Mode

The following instructions will configure the sensor and start a readout with the following settings:

- ***ABZ Mode***
- A/B Mode
- Absolute Position Mode
- Ungated Z-pulse
- 4096 pulses per revolution

This is a concrete example of commands to be sent line by line to achieve the desired configuration.

```
// Initialization
09
12
0F 00 00 F0
0D 00 00 F2
14 01 00 01 E9
2B 01 00 02 D1

// Pre-read
1D 03 00 3F 00 01 9F
1D 03 00 40 00 01 9E
1D 03 00 41 00 01 9D
1D 03 00 42 00 01 9C
1D 03 00 43 00 01 9B
1D 03 00 44 00 01 9A
1D 03 00 45 00 01 99
1D 03 00 46 00 01 98
1D 03 00 47 00 01 97
1D 03 00 48 00 01 96
1D 03 00 49 00 01 95
1D 03 00 4A 00 01 94
1D 03 00 4B 00 01 93
1D 03 00 4C 00 01 92
1D 03 00 4D 00 01 91

// Writing
1E 04 00 3F 00 19 00 85
1E 04 00 40 00 FF 0F 8F
1E 04 00 41 00 06 00 96
1E 04 00 42 00 00 00 9B
1E 04 00 43 00 00 00 9A
1E 04 00 44 00 00 00 99
1E 04 00 45 00 00 00 98
1E 04 00 46 00 00 00 97
1E 04 00 47 00 00 00 96
1E 04 00 48 00 00 20 75
1E 04 00 49 00 06 33 5B
1E 04 00 4A 00 00 00 93
1E 04 00 4B 00 14 00 7E
1E 04 00 4C 00 E6 0C 9F
1E 04 00 4D 00 44 B0 9C

// Apply Configuration with a Soft Reset
2A 01 00 01 D3

// Recheck
1D 03 00 3F 00 01 9F
1D 03 00 40 00 01 9E
1D 03 00 41 00 01 9D
1D 03 00 42 00 01 9C
1D 03 00 43 00 01 9B
1D 03 00 44 00 01 9A
1D 03 00 45 00 01 99
1D 03 00 46 00 01 98
1D 03 00 47 00 01 97
1D 03 00 48 00 01 96
1D 03 00 49 00 01 95
1D 03 00 4A 00 01 94
1D 03 00 4B 00 01 93
1D 03 00 4C 00 01 92
1D 03 00 4D 00 01 91

// Start the configured ABZ
2C 09 00 01 03 00 00 00 FF 0F 0F 00 A9

// Stop
2D 00 00 D2
```

2. Configuring UVW Mode

The following instructions will configure the sensor and start a readout with the following settings:

- ***UVW Mode***
- 16 pole pairs

This is a concrete example of commands to be sent line by line to achieve the desired configuration.

```
// Initialization
09
12
0F 00 00 F0
0D 00 00 F2
14 01 00 01 E9
2B 01 00 02 D1

// Pre-read
1D 03 00 3F 00 01 9F
1D 03 00 40 00 01 9E
1D 03 00 41 00 01 9D
1D 03 00 42 00 01 9C
1D 03 00 43 00 01 9B
1D 03 00 44 00 01 9A
1D 03 00 45 00 01 99
1D 03 00 46 00 01 98
1D 03 00 47 00 01 97
1D 03 00 48 00 01 96
1D 03 00 49 00 01 95
1D 03 00 4A 00 01 94
1D 03 00 4B 00 01 93
1D 03 00 4C 00 01 92
1D 03 00 4D 00 01 91

// Writing
1E 04 00 3F 00 1A 00 84
1E 04 00 40 00 FF 0F 8F
1E 04 00 41 00 06 00 96
1E 04 00 42 00 00 00 9B
1E 04 00 43 00 00 00 9A
1E 04 00 44 00 00 00 99
1E 04 00 45 00 00 00 98
1E 04 00 46 00 00 00 97
1E 04 00 47 00 00 00 96
1E 04 00 48 00 00 20 75
1E 04 00 49 00 06 33 5B
1E 04 00 4A 00 00 00 93
1E 04 00 4B 00 14 00 7E
1E 04 00 4C 00 E6 0C 9F
1E 04 00 4D 00 72 B0 6E

// Apply Configuration with a Soft Reset
2A 01 00 01 D3

// Recheck
1D 03 00 3F 00 01 9F
1D 03 00 40 00 01 9E
1D 03 00 41 00 01 9D
1D 03 00 42 00 01 9C
1D 03 00 43 00 01 9B
1D 03 00 44 00 01 9A
1D 03 00 45 00 01 99
1D 03 00 46 00 01 98
1D 03 00 47 00 01 97
1D 03 00 48 00 01 96
1D 03 00 49 00 01 95
1D 03 00 4A 00 01 94
1D 03 00 4B 00 01 93
1D 03 00 4C 00 01 92
1D 03 00 4D 00 01 91

// Start the configured UVW
2C 09 00 02 03 00 00 00 FF 0F 0F 00 A8

// Stop
2D 00 00 D2
```

3. Configuring SPI Mode

The following instructions will configure the sensor and start a readout with the following settings:

- ***SPI Mode***

This is a concrete example of commands to be sent line by line to achieve the desired configuration.

```
// Initialization
09
12
0F 00 00 F0
0D 00 00 F2
14 01 00 01 E9
2B 01 00 02 D1

// Pre-read
1D 03 00 3F 00 01 9F
1D 03 00 40 00 01 9E
1D 03 00 41 00 01 9D
1D 03 00 42 00 01 9C
1D 03 00 43 00 01 9B
1D 03 00 44 00 01 9A
1D 03 00 45 00 01 99
1D 03 00 46 00 01 98
1D 03 00 47 00 01 97
1D 03 00 48 00 01 96
1D 03 00 49 00 01 95
1D 03 00 4A 00 01 94
1D 03 00 4B 00 01 93
1D 03 00 4C 00 01 92
1D 03 00 4D 00 01 91

// Writing
1E 04 00 3F 00 18 00 86
1E 04 00 40 00 FF 0F 8F
1E 04 00 41 00 06 00 96
1E 04 00 42 00 00 00 9B
1E 04 00 43 00 00 00 9A
1E 04 00 44 00 00 00 99
1E 04 00 45 00 00 00 98
1E 04 00 46 00 00 00 97
1E 04 00 47 00 00 00 96
1E 04 00 48 00 00 20 75
1E 04 00 49 00 06 33 5B
1E 04 00 4A 00 00 00 93
1E 04 00 4B 00 14 00 7E
1E 04 00 4C 00 E6 0C 9F
1E 04 00 4D 00 56 B0 8A

// Apply Configuration with a Soft Reset
2A 01 00 01 D3

// Recheck
1D 03 00 3F 00 01 9F
1D 03 00 40 00 01 9E
1D 03 00 41 00 01 9D
1D 03 00 42 00 01 9C
1D 03 00 43 00 01 9B
1D 03 00 44 00 01 9A
1D 03 00 45 00 01 99
1D 03 00 46 00 01 98
1D 03 00 47 00 01 97
1D 03 00 48 00 01 96
1D 03 00 49 00 01 95
1D 03 00 4A 00 01 94
1D 03 00 4B 00 01 93
1D 03 00 4C 00 01 92
1D 03 00 4D 00 01 91

// Start the configured SPI
2C 09 00 00 03 00 00 00 FF 0F 0F 00 AA

// Stop
2D 00 00 D2
```

4. Configuring PWM Mode

The following instructions will configure the sensor and start a readout with the following settings:

- ***PWM Mode***

- 2500 Hz PWM frequency
- Synchronisation edge: Rising Edge

This is a concrete example of commands to be sent line by line to achieve the desired configuration.

```
// Initialization
09
12
0F 00 00 F0
0D 00 00 F2
14 01 00 01 E9
2B 01 00 02 D1

// Pre-read
1D 03 00 3F 00 01 9F
1D 03 00 40 00 01 9E
1D 03 00 41 00 01 9D
1D 03 00 42 00 01 9C
1D 03 00 43 00 01 9B
1D 03 00 44 00 01 9A
1D 03 00 45 00 01 99
1D 03 00 46 00 01 98
1D 03 00 47 00 01 97
1D 03 00 48 00 01 96
1D 03 00 49 00 01 95
1D 03 00 4A 00 01 94
1D 03 00 4B 00 01 93
1D 03 00 4C 00 01 92
1D 03 00 4D 00 01 91

// Writing
1E 04 00 3F 00 64 00 3A
1E 04 00 40 00 FF 0F 8F
1E 04 00 41 00 06 00 96
1E 04 00 42 00 00 00 9B
1E 04 00 43 00 00 00 9A
1E 04 00 44 00 00 00 99
1E 04 00 45 00 00 00 98
1E 04 00 46 00 00 00 97
1E 04 00 47 00 00 00 96
1E 04 00 48 00 00 20 75
1E 04 00 49 00 06 33 5B
1E 04 00 4A 00 00 00 93
1E 04 00 4B 00 14 00 7E
1E 04 00 4C 00 E6 0C 9F
1E 04 00 4D 00 3D B0 A3

// Apply Configuration with a Soft Reset
2A 01 00 01 D3

// Recheck
1D 03 00 3F 00 01 9F
1D 03 00 40 00 01 9E
1D 03 00 41 00 01 9D
1D 03 00 42 00 01 9C
1D 03 00 43 00 01 9B
1D 03 00 44 00 01 9A
1D 03 00 45 00 01 99
1D 03 00 46 00 01 98
1D 03 00 47 00 01 97
1D 03 00 48 00 01 96
1D 03 00 49 00 01 95
1D 03 00 4A 00 01 94
1D 03 00 4B 00 01 93
1D 03 00 4C 00 01 92
1D 03 00 4D 00 01 91

// Start the configured PWM
2C 09 00 03 04 01 00 00 FF 0F 0F 00 A5

// Stop
2D 00 00 D2
```

Important Considerations

⚠️ Two-Stage Process Required:

- **Stage 1** writes configuration to the sensor's volatile memory registers (all 15 registers: 63-77)
- **Stage 2** configures the MCU to capture and forward the interface signals
- Both stages are required for proper operation
- Skipping Stage 1 will only work if sensor already has correct CMTP configuration

⚠️ **Design Step Configuration is Mandatory:**

- After entering test mode, you **must** execute `CMD_SET_DESIGN_STEP (0x2B)` before any register operations
- This configures the MCU's register map layout for the specific sensor variant
- Failure to set design step will cause register read/write failures

⚠️ **Parameter Matching:**

- Parameters in Stage 2 (`CMD_START_IF_READOUT`) must match Stage 1 register configuration
- Example: `abz_uvw_pulses_pp` in register 64 must match ABZ pulses in `CMD_START_IF_READOUT`
- Mismatched parameters cause incorrect signal interpretation by the MCU

⚠️ **Volatile vs. Permanent:**

- All configuration changes in this workflow are temporary (written to VM only)
- Changes are lost on power cycle or sensor reset
- To make changes permanent, write to CMTP memory (see CMTP Programming Workflow)

⚠️ **CRC Calculation:**

- CRC must be calculated over **29 bytes total**:
  - Registers 63-76: All 28 bytes (14 registers × 2 bytes each, little-endian)
  - Register 77: Upper byte only (bits 15:8), 1 byte
- Register 77 bits 7:0 contains the calculated CRC result
- Algorithm: CRC8 SAE J1850 with polynomial 0x1D and seed 0xAA
- Process bytes in little-endian order (LSB first for each 16-bit word)
- The CRC covers all configuration data including the upper byte of register 77, but excludes the CRC field itself
- Incorrect CRC will cause sensor malfunction if CRC checking is enabled

Troubleshooting

Issue: Interface doesn't work after completing both stages

- **Cause**: Mismatch between Stage 1 VM configuration and Stage 2 MCU parameters
- **Solution**: Verify register 64 `abz_uvw_pulses_pp` matches `CMD_START_IF_READOUT` pulses parameter, verify `if_mode` matches interface mode

Issue: No async packets received after Stage 2

- **Cause**: Sensor still in test mode, or MCU not configured
- **Solution**: Ensure sensor exited test mode in Step 11, verify `CMD_START_IF_READOUT` sent correctly

Issue: Incorrect signal behavior (wrong resolution, frequency, Z-pulse, etc.)

- **Cause**: Register bitfield values calculated incorrectly
- **Solution**: Verify bitfield calculations, check bit positions and lengths in register map, verify bit shift operations

Issue: Sensor doesn't respond after soft reset

- **Cause**: Invalid register values causing sensor fault
- **Solution**: Power cycle evaluation board, verify register values are within valid ranges, check CRC calculation

Issue: CRC errors reported by sensor

- **Cause**: Incorrect CRC calculation or algorithm
- **Solution**: Verify CRC8 SAE J1850 implementation with seed 0xAA, ensure all 15 registers included, check byte order (little-endian)

Issue: Configuration reverts after power cycle

- **Cause**: Changes only written to volatile memory, not CMTP
- **Solution**: This is expected behavior for VM. To persist, write configuration to CMTP (see CMTP Programming Workflow)

Issue: Register write appears successful but configuration doesn't apply

- **Cause**: Soft reset not performed after writing registers
- **Solution**: Always execute Step 10 (soft reset) after modifying registers to reload VM

Best Practices

1. **Read before write** - Always read current register values to preserve unmodified bitfields
2. **Verify after write** - Read back registers after writing to confirm correct values.
3. **Calculate CRC carefully** - Use proven CRC8 SAE J1850 implementation, verify with test vectors
4. **Match parameters between stages** - Ensure Stage 1 (sensor VM) and Stage 2 (MCU config) parameters are identical
5. **Test in VM first** - Thoroughly test in volatile memory before writing to CMTP
6. **Document register values** - Keep track of all bitfield values and register calculations
7. **Use correct design step** - Set design step with `CMD_SET_DESIGN_STEP (0x2B)` before register access
8. **Handle errors** - Check SPI response frames for errors after each write
9. **Preserve bitfields** - Only modify target bitfields, preserve all others
10. **Test incrementally** - Configure and test one interface mode at a time

---

CMTP Programming Workflow

This section provides a step-by-step workflow for programming CMTP (OTP) memory on the TLx49012 sensor.

Overview

CMTP programming involves reading existing memory, modifying specific values, calculating a new CRC, writing the updated data, and verifying the results. This workflow ensures data integrity throughout the programming process.

Prerequisites

- Evaluation kit connected to PC via USB
- FTDI drivers installed (COM port available)
- Sensor properly connected to evaluation board
- Understanding of CMTP memory structure and CRC calculation

CMTP Programming Flow

Step 1: Connect Programmer to PC

1. Connect the evaluation kit programmer to your PC via USB
2. Wait for the device to enumerate as a COM port
3. Identify the COM port number from Device Manager (e.g., COM3, COM5)

Step 2: Configure Serial Port

Open the serial port with the following configuration:

- **Baud Rate**: 3,000,000
- **Data Bits**: 8
- **Stop Bits**: 1
- **Parity**: None

Step 3: Execute Bootloader Commands

Perform the bootloader handshake sequence to transition to firmware mode.

3.1 - Send Echo Command: Send `0x09` and wait for response `0x20` (bootloader acknowledgment).

3.2 - Switch to Firmware Mode: Send `0x12` and wait for response `0x20`. MCU will restart into firmware mode.

Note: After restart, wait at least 100ms for firmware initialization.

Step 4: Enable Sensor Supply

Enable the sensor power supply. Send `0F 00 00 F0` (CMD_IO_ENABLE_SENSOR_SUPPLY) and wait for response `0F 00 00 F0`.

Step 5: Enable SPI Interface

Enable the SPI interface. Send `0D 00 00 F2` (CMD_SPI_ENABLE) and wait for response `0D 00 00 F2`.

Step 6: Set Design Step

⚠️ **MANDATORY**: Configure the MCU for the correct design step. Send `2B 01 00 02 D1` (CMD_SET_DESIGN_STEP - Design step B12, value 2) and wait for response `2B 00 00 D4`. This configures the MCU's register map and memory layout for the sensor variant.

Note: This step is required before any register read/write operations.

Step 7: Enter Test Mode

Before any memory operations, enter test mode. Send `14 01 00 01 E9` (CMD_SPI_ENTER_TEST_MODE) and wait for response `14 04 00 [4 bytes SPI frame] [checksum]`. Sensor is now in test mode.

Step 8: Configure OTP Status Register

Write to OTP status register (register 253 / 0xFD) to configure CMTP access. Send `1E 04 00 FD 00 0D 00 D3` (CMD_SPI_WRITE_REG_NEXT_FRAME) and wait for response `1E 04 00 [4 bytes SPI frame] [checksum]`. This configures CMTP access parameters.

Step 9: Read Whole Memory (Pre-Programming)

Read the current CMTP memory contents. Send `1C 00 00 E3` (CMD_SPI_READ_CMTP) and wait for response `1C 78 00 [60 words = 120 bytes] [checksum]`. Parse the 60 words (120 bytes) from the response and store memory contents for analysis and CRC calculation.

Step 10: Read Access Rights

Check remaining programming cycles. Reconfigure OTP status register: `1E 04 00 FD 00 0D 00 D3`. Send `28 00 00 D7` (CMD_READ_ALL_ACCESS_RIGHTS) and wait for response `28 78 00 [60 words] [checksum]`. For each word, extract remaining cycles: `cycles = (word >> 8) & 0x1F`. Verify target addresses have available programming cycles.

Important: Check access rights before programming to ensure addresses have remaining programming cycles.

Step 11: Disable Firmware Execution

Write to register 199 (0xC7) to disable firmware execution during programming. Send `1E 04 00 C7 00 00 40 D6` (CMD_SPI_WRITE_REG_NEXT_FRAME) and wait for response `1E 04 00 [4 bytes SPI frame]`

`[checksum]`. This prevents firmware interference during CMTP programming.

Step 12: Enable Programming Voltage

Enable V_PROG for CMTP programming. Send `11 00 00 EE` (CMD_IO_ENABLE_SENSOR_V_PROG) and wait for response `11 00 00 EE`. V_PROG is now enabled. Wait for V_PROG to settle (2.5 milliseconds) or use the command CMD_WAIT_V_PROG_SETTLE to check if V_PROG is ready before proceeding.

Step 13: Configure CMTP Write Enable

Write to OTP status register to enable CMTP write operations. Send `1E 04 00 FD 00 17 00 C9` (CMD_SPI_WRITE_REG_NEXT_FRAME) and wait for response `1E 04 00 [4 bytes SPI frame] [checksum]`. This enables CMTP memory write access.

Step 14: Set Cold Temperature Programming mode by writing value 0 to register 336 (0x150)

Write to register 336 (0x150) the value 0 (0x0000). Send `1E 04 00 50 01 00 00 8C` (CMD_SPI_WRITE_REG_NEXT_FRAME) and wait for response `1E 04 00 [4 bytes SPI frame] [checksum]`.

Step 15: Write CMTP LUT Block

Program LUT (Look-Up Table) data (addresses 256-288 / 0x100-0x120):

15.1 - Write LUT Data: For each address from 256 (0x100) to 288 (0x120), send `29 04 00 [addr_low] [addr_high] [value_low] [value_high] [checksum]` and wait for response `29 03 00 [dummy] [status_254] [status_255] [checksum]`. Verify status bytes are `0x00 0x00` (no error).

Example sequence:

```
29 04 00 00 01 00 00 D1  (Address 256 / 0x100)
29 04 00 01 01 00 10 C0  (Address 257 / 0x101)
29 04 00 02 01 00 20 AF  (Address 258 / 0x102)
...continue for all LUT addresses...
29 04 00 20 01 C7 C7 23  (Address 288 / 0x120 - includes CRC)
```

Note: The last address in each block contains the CRC for that block.

Step 16: Write CMTP User Configuration Block

Program user configuration data (addresses 289-303 / 0x121-0x12F):

16.1 - Write User Configuration Registers: For each address from 289 (0x121) to 303 (0x12F), send `29 04 00 [addr_low] [addr_high] [value_low] [value_high] [checksum]` and wait for response `29 03 00 [dummy] [status_254] [status_255] [checksum]`. Verify status bytes are `0x00 0x00` (no error).

Example sequence:

```
29 04 00 21 01 1D 00 93  (Address 289 / 0x121)
29 04 00 22 01 FF 0F A1  (Address 290 / 0x122)
29 04 00 23 01 06 00 A8  (Address 291 / 0x123)
...continue for all user config addresses...
29 04 00 2F 01 0C B0 E6  (Address 303 / 0x12F - includes CRC)
```

Step 17: Disable Programming Voltage

Disable V_PROG after programming complete. Send `12 00 00 ED` (CMD_IO_DISABLE_SENSOR_V_PROG) and wait for response `12 00 00 ED`.

Step 18: Reset the sensor and reload from NVME by entering Test Mode (for Verification)

Re-enter test mode to verify programming. Send `14 01 00 01 E9` (CMD_SPI_ENTER_TEST_MODE) and wait for response `14 04 00 [4 bytes SPI frame] [checksum]`. Sensor is back in test mode for verification.

Step 19: Configure OTP Status for Read

Configure OTP status register for reading. Send `1E 04 00 FD 00 0D 00 D3` (CMD_SPI_WRITE_REG_NEXT_FRAME) and wait for response.

Step 20: Read Whole Memory (Post-Programming Verification)

Verify that programming was successful by reading back the memory. Send `1C 00 00 E3` (CMD_SPI_READ_CMTP) and wait for response `1C 78 00 [60 words] [checksum]`. Parse and compare with expected values. Verify all programmed addresses match expected data.

Step 21: Verify Access Rights (Final Check)

Read access rights to confirm programming cycles were consumed. Send `1E 04 00 FD 00 0D 00 D3` (Configure OTP status). Send `28 00 00 D7` (CMD_READ_ALL_ACCESS_RIGHTS) and wait for response `28 78 00 [60 words] [checksum]`. Verify programming cycles decremented for programmed addresses.

Verification Checklist:

- ✓ All programmed values match expected values
- ✓ CRCs are correct for all blocks (User Config and LUT)
- ✓ No unexpected changes in other memory locations
- ✓ Programming cycles decremented correctly
- ✓ Diagnostic registers show no errors
- ✓ Sensor functionality verified in normal operation mode

Step 22: Additional check (VM)

The Volatile Memory should have been already loaded with the new CMTP values by this point. Additional Volatile Memory check can be performed here by reading registers 63-77 and verifying their values and CRC match the expected configuration for User registers.

Important Considerations

⚠️ **Programming Cycles:**

- Each CMTP address has a limited number of programming cycles (0-5)
- Check access rights (CMD 0x28) before programming
- Plan programming carefully to avoid exhausting cycles

⚠️ **CRC Calculation:**

- The CRC algorithm must match the sensor specification
- TLx49012 uses CRC8 SAE J1850 with seed 0xAA (polynomial: 0x1D)
- **For User Configuration Registers (VM workflow)**: Single CRC calculated over 29 bytes
  - Input: Registers 63-76 (28 bytes) + Upper byte of register 77 (1 byte)
  - Register 63-76: All 16 bits of each register (2 bytes × 14 registers = 28 bytes)
  - Register 77: Only the upper byte (bits 15:8), excluding the CRC field itself
  - Total: 29 bytes in little-endian order
  - Output: 8-bit CRC stored in register 77 bits 7:0
- **For CMTP Memory**: Four separate CRCs for different memory blocks (see CMTP Programming Workflow)
- Incorrect CRC will cause sensor malfunction

⚠️ **Address Range:**

- Standard CMTP range: 256-315 (0x0100-0x013B)
- Only program addresses within the valid range
- Respect design step-specific address variations

⚠️ **Programming Voltage:**

- Only enable V_PROG during programming operations
- Always wait for V_PROG to settle before writing
- Disable V_PROG immediately after programming
- Prolonged V_PROG application may damage the sensor

⚠️ **Error Handling:**

- Always check status registers after each write operation
- Non-zero status values indicate errors
- Stop programming immediately if errors occur

⚠️ **Verification:**

- Always verify programming by reading back memory after completion
- Test sensor functionality in normal operation mode
- Keep backup of original memory contents

⚠️ **One-Time Programmable:**

- CMTP is OTP memory - values cannot be erased
- Bits can only be programmed from 1 to 0
- Plan programming carefully and verify before execution

⚠️ **Backup:**

- Always save the original memory contents before programming
- Document all changes made
- Maintain programming history for traceability

Complete CMTP Examples:

⚠️ **To consider:**

- These examples have been validated for the TLE49012-S0001 and TLI49012-E0001.
- If you are programming a different sensor, please consult the User Guide to identify the correct if_mode bitfield configuration, as these values vary by sensor type.

Initialization and Pre-Write Steps

The following commands are common initialization and pre-read steps for all interface modes (ABZ, UVW, PWM, SPI). These steps are necessary for a successful programming of the CMTP.

```
// Initialization
09
12
0F 00 00 F0
0D 00 00 F2
14 01 00 01 E9
2B 01 00 02 D1

// Configure sensor for CMTP access
0D 00 00 F2
14 01 00 01 E9
17 02 00 78 01 6D

// Read whole CMTP memory
1E 04 00 FD 00 0D 00 D3
1C 00 00 E3

// Read acces rights + programming cycles
1E 04 00 FD 00 0D 00 D3
28 00 00 D7

// Firmware execution disable
1E 04 00 C7 00 00 40 D6

// Enable Programming Voltage
11 00 00 EE

// Disable CMTP, Write enable, etc
1E 04 00 FD 00 17 00 C9

// Set Cold Temperature Programming mode by writing value 0 to register 336 (0x150)
1E 04 00 50 01 00 00 8C
```

1. Configuring ABZ Mode through the CMTP

The following instructions will configure the sensor and start a readout with the following settings:

- ***ABZ Mode***
- A/B Mode
- Absolute Position Mode
- Ungated Z-pulse
- 4096 pulses per revolution

This is a concrete example of commands to be sent line by line to achieve the desired configuration in the USER part of the CMTP.

```
// CMTP setup - User Section
29 04 00 21 01 19 00 97
29 04 00 22 01 FF 0F A1
29 04 00 23 01 06 00 A8
29 04 00 24 01 00 00 AD
29 04 00 25 01 00 00 AC
29 04 00 26 01 00 00 AB
29 04 00 27 01 00 00 AA
29 04 00 28 01 00 00 A9
29 04 00 29 01 00 00 A8
29 04 00 2A 01 00 20 87
29 04 00 2B 01 06 33 6D
29 04 00 2C 01 00 00 A5
29 04 00 2D 01 14 00 90
29 04 00 2E 01 E6 0C B1
29 04 00 2F 01 44 B0 AE
```

2. Configuring UVW Mode through the CMTP

The following instructions will configure the sensor and start a readout with the following settings:

- ***UVW Mode***
- 16 pole pairs

This is a concrete example of commands to be sent line by line to achieve the desired configuration in the USER part of the CMTP.

```
// CMTP setup - User Section
29 04 00 21 01 1A 00 96
29 04 00 22 01 FF 0F A1
29 04 00 23 01 06 00 A8
29 04 00 24 01 00 00 AD
29 04 00 25 01 00 00 AC
29 04 00 26 01 00 00 AB
29 04 00 27 01 00 00 AA
29 04 00 28 01 00 00 A9
29 04 00 29 01 00 00 A8
29 04 00 2A 01 00 20 87
29 04 00 2B 01 06 33 6D
29 04 00 2C 01 00 00 A5
29 04 00 2D 01 14 00 90
29 04 00 2E 01 E6 0C B1
29 04 00 2F 01 72 B0 80
```

3. Configuring SPI Mode through the CMTP

The following instructions will configure the sensor and start a readout with the following settings:

- ***SPI Mode***

This is a concrete example of commands to be sent line by line to achieve the desired configuration in the USER part of the CMTP.

```
// CMTP setup - User Section
29 04 00 21 01 18 00 98
29 04 00 22 01 FF 0F A1
29 04 00 23 01 06 00 A8
29 04 00 24 01 00 00 AD
29 04 00 25 01 00 00 AC
29 04 00 26 01 00 00 AB
29 04 00 27 01 00 00 AA
29 04 00 28 01 00 00 A9
29 04 00 29 01 00 00 A8
29 04 00 2A 01 00 20 87
29 04 00 2B 01 06 33 6D
29 04 00 2C 01 00 00 A5
29 04 00 2D 01 14 00 90
29 04 00 2E 01 E6 0C B1
29 04 00 2F 01 56 B0 9C
```

4. Configuring PWM Mode through the CMTP

The following instructions will configure the sensor and start a readout with the following settings:

- ***PWM Mode***

- 2500 Hz PWM frequency
- Synchronisation edge: Rising Edge

This is a concrete example of commands to be sent line by line to achieve the desired configuration in the USER part of the CMTP.

```
// CMTP setup - User Section
29 04 00 21 01 64 00 4C
29 04 00 22 01 FF 0F A1
29 04 00 23 01 06 00 A8
29 04 00 24 01 00 00 AD
29 04 00 25 01 00 00 AC
29 04 00 26 01 00 00 AB
29 04 00 27 01 00 00 AA
29 04 00 28 01 00 00 A9
29 04 00 29 01 00 00 A8
29 04 00 2A 01 00 20 87
29 04 00 2B 01 06 33 6D
29 04 00 2C 01 00 00 A5
29 04 00 2D 01 14 00 90
29 04 00 2E 01 E6 0C B1
29 04 00 2F 01 3D B0 B5
```

LUT writing section

While not necessary, the LUT section can also be programmed with custom values if desired. Below is an example of how to program the LUT section of the CMTP with custom values. The actual values will depend on the specific application and requirements. The CRC is already calculated and integrated for this example.

```
// Lut Part
29 04 00 00 01 00 00 D1
29 04 00 01 01 00 10 C0
29 04 00 02 01 00 20 AF
29 04 00 03 01 00 30 9E
29 04 00 04 01 00 40 8D
29 04 00 05 01 00 50 7C
29 04 00 06 01 00 60 6B
29 04 00 07 01 00 70 5A
29 04 00 08 01 00 80 49
29 04 00 09 01 00 90 38
29 04 00 0A 01 00 A0 27
29 04 00 0B 01 00 B0 16
29 04 00 0C 01 00 C0 05
29 04 00 0D 01 00 D0 F4
29 04 00 0E 01 00 E0 E3
29 04 00 0F 01 00 F0 D2
29 04 00 10 01 00 00 C1
29 04 00 11 01 00 10 B0
29 04 00 12 01 00 20 9F
29 04 00 13 01 00 30 8E
29 04 00 14 01 00 40 7D
29 04 00 15 01 00 50 6C
29 04 00 16 01 00 60 5B
29 04 00 17 01 00 70 4A
29 04 00 18 01 00 80 39
29 04 00 19 01 00 90 28
29 04 00 1A 01 00 A0 17
29 04 00 1B 01 00 B0 06
29 04 00 1C 01 00 C0 F5
29 04 00 1D 01 00 D0 E4
29 04 00 1E 01 00 E0 D3
29 04 00 1F 01 00 F0 C2
29 04 00 20 01 C7 C7 23
```

Post-Write Steps and Check

After writing the CMTP memory, the following steps should be performed to verify that the programming was successful and that the sensor is functioning correctly with the new configuration.

```
// Disable Programming Voltage
12 00 00 ED

// Reset the sensor to load written values from CMTP into VM
0D 00 00 F2
14 01 00 01 E9

// Read whole CMTP memory
1E 04 00 FD 00 0D 00 D3
1C 00 00 E3

// Read acces rights + programming cycles
1E 04 00 FD 00 0D 00 D3
28 00 00 D7
```

Aditional checks (VM)

To further check if the values have been written correctly, the volatile memory can be read and verified against the expected configuration. This step is optional but can provide additional confidence in the programming process. The steps required for this can be found in the VM programming workflow, specifically in the “Recheck” section after applying the configuration with a soft reset.

Troubleshooting

Issue: Write command returns non-zero status

- **Cause**: Insufficient programming cycles, invalid address, or hardware fault
- **Solution**: Check access rights, verify address is in valid range, check V_PROG level

Issue: Verification shows incorrect values

- **Cause**: Programming voltage issue, communication errors, or hardware fault
- **Solution**: Check hardware connections, verify checksums, retry programming with fresh device

Issue: Sensor not responding after programming

- **Cause**: Incorrect CRC or corrupted configuration
- **Solution**: Verify CRC calculation, check memory block boundaries, test with known-good configuration

Issue: Cannot enter test mode

- **Cause**: Sensor in abnormal state, hardware connection issue, or power problem
- **Solution**: Power cycle the evaluation board, verify USB connection, check COM port configuration